



Courses : Advanced Web Programming (PWL)
Courses : D4 – Informatics Engineering / D4 – Business Information Systems
Semester : 4 (four) / 5 (five)
The meeting : 8 (eight)

JOBSHEET 08

Import and Export Files to PDF and Excel on Laravel

In accordance with **PWL.pdf Case Study**. So our Laravel 10 project is still the same as using **the PWL_POS repository**.

We will **use PWL_POS** project until the 12th meeting, as a project that we will learn

A. File Upload (import data) on Laravel

Uploading files is one of the essential features that are often found in various website projects. This feature provides the ability for users to move files from their local computer device into a web application server environment. This process involves users selecting files from their local system, such as images, documents, or other media, which are then uploaded through the form provided on the website. Once the file has been successfully uploaded, the server will store it in a specific directory that has been predefined by the developer. This storage can be on a local server, *cloud storage*, or even in a content distribution network (CDN) to ensure optimal performance and availability. With this file upload feature, users can dynamically enrich the website's content, while developers can ensure that uploaded files are properly managed, secure, and in accordance with the app's policies.

Uploading files is not only used to add media or documents to a website, but also has an important role in the data management process, especially for *importing* data into databases in Laravel based applications. This feature allows users to upload files containing data in formats such as CSV or Excel, which are then processed by the application to be included in the database.

This process usually begins with the user selecting a file containing the data from their local computer and uploading it to the server through a form provided in the web application.



Once the file is successfully uploaded, Laravel will process the file using a package like Maatwebsite Excel, which is specifically designed to handle data import from Excel or CSV files. In this jobsheet, we will learn how to upload the files used for the data input process into the database (import) using Laravel.

Practicum 1 – Implementation of File Upload for data import:

1. We open the database that we have created before, and we check the **m_kategori** and **m_barang** tables

kategori_id	kategori_kode	kategori_nama	created_at	updated_at
1	SBK	Sembako	2024-08-16 02:57:53	NULL
2	SNK	Makanan Ringan	2024-08-16 02:57:53	NULL
3	MND	Peralatan Mandi	2024-08-16 02:57:53	NULL
4	BAY	Keperluan Bayi	2024-08-16 02:57:53	NULL
5	MNM	Minuman	2024-08-16 02:57:53	NULL

barang_id	kategori_id	barang_kode	barang_nama	harga_beli	harga_jual	created_at	updated_at
1	1	SBK-001	Beras Cap Jago (5kg)	65000	68000	2024-08-19 20:07:06	NULL
2	1	SBK-002	Beras Bramo Cap Lele	80000	83000	2024-08-19 20:07:06	NULL
3	2	SNK-001	Happy Toss	10500	11000	2024-08-19 20:07:06	NULL
4	2	SNK-002	Oreo	7200	7800	2024-08-19 20:07:06	NULL
5	3	MND-001	Sabun Lifebouy	4250	5000	2024-08-19 20:07:06	NULL
6	3	MND-002	Pasta Gigi Pepsoden	6750	7500	2024-08-19 20:07:06	NULL
7	4	BAY-001	Susu SGM Coklat 900gr	92500	95000	2024-08-19 20:07:06	NULL
8	4	BAY-002	Popok Mamy Poko	56000	58000	2024-08-19 20:07:06	NULL
9	5	MNM-001	Aqua 600ml	3700	4500	2024-08-19 20:07:06	NULL
10	5	MNM-002	Le Mineral	3500	4000	2024-08-19 20:07:06	NULL

2. Here we will try to enter the data into our system in bulk. We can use excel files to try to enter the data of the goods into our system. We create an excel file template that we will use to import data items.

The screenshot shows an Excel spreadsheet titled 'template_barang.xlsx'. The spreadsheet has columns for 'kategori_id', 'barang_kode', 'barang_nama', 'harga_beli', and 'harga_jual'. The data is organized into rows, with the first row containing the headers and subsequent rows containing sample data for various goods.

	A	B	C	D	E	F
1	kategori_id	barang_kode	barang_nama	harga_beli	harga_jual	
2		1 SBK-003	Telur Omega (10 butir)	22000	25000	
3		2 SNK-003	Sari Roti	11500	12500	
4		3 MND-003	Shampo Pantene	17500	18500	
5		4 BAY-003	Baju Bayi 2th	89000	92500	
6		5 MNM-003	Cleo 600ml	3750	4300	
7						



For example, we want to import 5 item data at once. We save it to an excel file with the name `template_barang.xlsx`, and we save it in a `public` folder in our web project.

3. Next, we modify the view on the `goods/index.blade.php` to be able to add a button to add a form to upload for importing goods data

```
@extends('layouts.template')

@section('content')
    <div class="card">
        <div class="card-header">
            <h3 class="card-title">List of items</h3>
            <div class="card-tools">
                <button onclick="modalAction('{{ url('/goods/import') }}')" class="btn btn-info">Import Goods</button>
                <a href="{{ url('/item/create') }}" class="btn btn-primary">Add Data</a>
                <button onclick="modalAction('{{ url('/item/create_ajax') }}')" class="btn btn-success">Add Data (Ajax)</button>
            </div>
        </div>
        <div class="card-body">
            <!-- for Data filter -->
            <div id="filter" class="form-horizontal filter-date p-2 border-bottom mb-2">
                <div class="row">
                    <div class="col-md-12">
                        <div class="form-group form-group-sm row text-sm mb-0">
                            <label for="filter_date" class="col-md-1 col-form-label">Filter</label>
                            <div class="col-md-3">
                                <select name="filter_kategori" class="form-control form-control-sm filter_kategori">
                                    <option value="">- All -</option>
                                    @foreach($kategori as $l)
                                        <option value="{{ $l->kategori_id }}">{{ $l->kategori_nama }}</option>
                                    @endforeach
                                </select>
                                <small class="form-text text-muted">Item Category</small>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
            @if(session('success'))
                <div class="alert alert-success">{{ session('success') }}</div>
            @endif
            @if(session('error'))
                <div class="alert alert-danger">{{ session('error') }}</div>
            @endif
            <table class="table table-bordered table-sm table-striped table-hover" id="table-item">
                <thead>
                    <tr><th>No</th><th>Item Code</th><th>Item Code</th><th>Purchase Price</th><th>Selling Price</th><th>Category</th><th>Action</th></tr>
                </thead>
                <tbody></tbody>
            </table>
        </div>
    </div>
    <div id="myModal" class="modal fade animate shake" tabindex="-1" data-backdrop="static" data-keyboard="false" data-width="75%"></div>

@endsection

@push('js')
```



```
<script>
    function modalAction(url = ''){
        $('#myModal').load(url,function(){
            $('#myModal').modal('show');
        });
    }

    var tableGoods;
    $(document).ready(function(){
        tableItem = $('#table-item'). DataTable({
            True,
            serverSide: true,
            Ajax: {
                "url": "{ url('item/list') }",
                "dataType": "json",
                "type": "POST",
                "data": function (d) {
                    d.filter_kategori = $('#filter_kategori').val();
                }
            },
            Columns: [{
                Date: "No_Urut",
                className: "text-center",
                width: "5%",
                Orderable: False.
                searchable: false
            },{
                Date: "barang_kode",
                className: "",
                width: "10%",
                orderable: true,
                Searchable: True
            },{
                Date: "barang_nama",
                className: "",
                width: "37%",
                orderable: true,
                searchable: true,
            },{
                Date: "harga_beli",
                className: "",
                width: "10%",
                orderable: true,
                searchable: false.
                render: function(data, type, row){
                    return new Intl.NumberFormat('id-ID').format(data);
                }
            },{
                Date: "harga_jual",
                className: "",
                width: "10%",
                orderable: true,
                searchable: false.
                render: function(data, type, row){
                    return new Intl.NumberFormat('id-ID').format(data);
                }
            },{
                Date: "kategori.kategori_nama",
                className: "",
                Size: "14%",
                orderable: true,
                searchable: false
            },{
                data: "action",
                className: "text-center",
                Size: "14%",
                Orderable: False.
            }
        ]
    })
</script>
```



```
searchable: false
}
]
});

$('#table-barang_filter input').unbind().bind().on('keyup', function(e){
    if(e.keyCode == 13){ enter key
    tableItem.search(this.value).draw();
    }
});

$('.filter_kategori').change(function(){
    tableItem.draw();
});
});
</script>
@endpush
```

4. Next, we create a view for the excel file upload/import form and download the `template_barang.xlsx` file. We name the file with the name of the `item/import.blade.php`

```
<form action="{{ url('/item/import_ajax') }}" method="POST" id="form-import"
    enctype="multipart/form-data">
    @csrf
    <div id="modal-master" class="modal-dialog modal-lg" role="document">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title" id="exampleModalLabel">Import Item Data</h5>
                <button type="button" class="close" data-dismiss="modal" aria-
label="Close"><span aria-hidden="true">&times;</span></button>
            </div>
            <div class="modal-body">
                <div class="form-group">
                    <label>Download Template</label>
                    <a href="{{ asset('template_barang.xlsx') }}" class="btn btn-info btn-
sm" download><i class="fa fa-file-excel"></i>Download</a>
                    <small id="error_kategori_id" class="error-text form-text text-
danger"></small>
                </div>
                <div class="form-group">
                    <label>Select File</label>
                    <input type="file" name="file_barang" id="file_barang" class="form-
control" required>
                    <small id="error-file_barang" class="error-text form-text text-
danger"></small>
                </div>
            </div>
            <div class="modal-footer">
                <button type="button" data-dismiss="modal" class="btn btn-
warning">Cancel</button>
                <button type="submit" class="btn btn-primary">Upload</button>
            </div>
        </div>
    </form>
    <script>
    $(document).ready(function() {
    $("#form-import").validate({
    Rules: {
    file_barang: {required: true, extension: ".xlsx"},
    },
    submitHandler: function(form) {
```



```
var formData = new FormData(form);    Make the form into FormData to handle
the file

$.ajax({
  URL: form.action,
  Type: form.method,
  data: formData,    Data sent in the form of FormData
  processData: false,    setting processData and contentType to false, to handle the file
  contentType: false,
  success: function(response) {
    if(response.status){    if successful
      $('#myModal').modal('hide');
      Swal.fire({
        icon: 'success',
        title: 'Successful',
        Text: response.message
      });
      tableItem.ajax.reload();    Reload datatable
    }else{    if there is an error
      $('.error-text').text('');
      $.each(response.msgField, function(prefix, val) {
        $('#error-'+prefix).text(val[0]);
      });
      Swal.fire({
        icon: 'error',
        title: 'An Error Occurred',
        Text: response.message
      });
    }
  }
});

return false;
},
errorElement: 'span',
errorPlacement: function (error, element) {
  error.addClass('invalid-feedback');
  element.closest('.form-group').append(error);
},
highlight: function (element, errorClass, validClass) {
  $(element).addClass('is-invalid');
},
unhighlight: function (element, errorClass, validClass) {
  $(element).removeClass('is-invalid');
}
});
});
</script>
```

5. Then we modify the `route/web.php` to accommodate the file upload process on the item menu

```
Route::get('/barang',[BarangController::class,'index']);
Route::post('/barang/list',[BarangController::class,'list']);
Route::get('/barang/create_ajax',[BarangController::class,'create_ajax']); // ajax form create
Route::post('/barang_ajax',[BarangController::class,'store_ajax']); // ajax store
Route::get('/barang/{id}/edit_ajax',[BarangController::class,'edit_ajax']); // ajax form edit
Route::put('/barang/{id}/update_ajax',[BarangController::class,'update_ajax']); // ajax update
Route::get('/barang/{id}/delete_ajax',[BarangController::class,'confirm_ajax']); // ajax form confirm
Route::delete('/barang/{id}/delete_ajax',[BarangController::class,'delete_ajax']); // ajax delete
Route::get('/barang/import',[BarangController::class,'import']); // ajax form upload excel
Route::post('/barang/import_ajax',[BarangController::class,'import_ajax']); // ajax import excel
```



6. To be able to read/write excel files, we need a library to read/write excel files. So we can use the [phpoffice/phpspreadsheet](#) library. We type the command in the terminal/CMD

```
composer require phpoffice/phpspreadsheet
```

7. Next we modify the [BarangController.php](#) file to process the data

```
<?php
namespace App\Http\Controllers;

use App\Models\LevelModel;
use App\Models\ItemModel;
use App\Models\CategoryModel;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades\Validator;
use PhpOffice\PhpSpreadsheet\IOFactory;
use Yajra\DataTables\Facades\DataTables;

class GoodsController extends Controller
{
    public function index (){
        $activeMenu = 'goods';
        $breadcrumb = (object) [
            'title' => 'Item Data',
            'list' => ['Home', 'Goods']
        ];

        $kategori = CategoryModel::select('kategori_id', 'kategori_nama')->get();
        return view('item.index', [
            'activeMenu' => $activeMenu,
            'breadcrumb' => $breadcrumb,
            'category' => $kategori
        ]);
    }

    public function list(Request $request)
    {
        $barang = ItemModel::select('barang_id', 'barang_kode', 'barang_nama', 'harga_beli',
            'harga_jual', 'kategori_id')->with('category');

        $kategori_id = $request->input('filter_kategori');
        if(!empty($kategori_id)){
            $barang->where('kategori_id', $kategori_id);
        }

        return DataTables::of($barang)
            ->addIndexColumn()
            ->addColumn('action', function ($barang) {
                add action column
                /*$btn = '<a href="'.url('/item/'. $barang->barang_id).'>' class="btn btn-
                info btn-sm">Detail</a>';
                $btn .= '<a href="'.url('/item/'. $barang->barang_id . '/edit').'" class="btn btn-warning
                btn-sm">Edit</a>';
                $btn .= '<form class="d-inline-block" method="POST" action="'.
                url('/item/'. $barang->barang_id).'>'
                . csrf_field() . method_field('DELETE') .
                '<button type="submit" class="btn btn-danger btn-sm" onclick="return confirm('\Are we going
                to delete this data?');">Delete</button></form>';*/
                $btn = '<button onclick="modalAction('\'.url('/item/'. $barang->barang_id .
                '/show ajax)').'\'" class="btn btn-info btn-sm">Detail</button>';
            })
            ->make();
    }
}
```



```
$btn .= '<button onclick="modalAction(\''.url('/item/' . $barang->barang_id .  
'/edit_ajax').'\')"' class="btn btn-warning btn-sm">Edit</button> ';  
$btn .= '<button onclick="modalAction(\''.url('/item/' . $barang->barang_id .  
'/delete_ajax').'\')"' class="btn btn-danger btn-sm">Delete</button> ';  
return $btn;  
})  
->rawColumns(['action']) contains html text  
->make(true);  
}  
  
Public function create_ajax()  
{  
$kategori = CategoryModel::select('kategori_id', 'kategori_nama')->get();  
return view('barang.create_ajax')->with('category', $kategori);  
}  
  
public function store_ajax(Request $request)  
{  
if($request->ajax() || $request->wantsJson()){  
$rules = [  
    'kategori_id' => ['required', 'integer', 'exists:m_kategori,kategori_id'],  
    'barang_kode' => ['required', 'min:3', 'max:20'],  
    'unique:m_barang,barang_kode'],  
    'barang_nama' => ['required', 'string', 'max:100'],  
    'harga_beli' => ['required', 'numeric'],  
    'harga_jual' => ['required', 'numeric'],  
];  
  
$validator = Validator::make($request->all(), $rules);  
if($validator->fails()){  
return response()->json([  
    'status' => false,  
    'message' => 'Validation Failed',  
    'msgField' => $validator->errors()  
]);  
}  
  
ItemModel::create($request->all());  
return response()->json([  
    'status' => true,  
    'message' => 'Data saved successfully'  
]);  
}  
redirect('/');  
}  
  
Public function edit_ajax($id)  
{  
$barang = ItemModel::find($id);  
$level = LevelModel::select('level_id', 'level_nama')->get();  
return view('barang.edit_ajax', ['item' => $barang, 'level' => $level]);  
}  
  
public function update_ajax(Request $request, $id)  
{  
Check if the request from Ajax  
if ($request->ajax() || $request->wantsJson()) {  
$rules = [  
    'kategori_id' => ['required', 'integer', 'exists:m_kategori,kategori_id'],  
    'barang_kode' => ['required', 'min:3', 'max:20'],  
    'unique:m_barang,barang_kode, '. $id .',barang_id'],  
    'barang_nama' => ['required', 'string', 'max:100'],  
    'harga_beli' => ['required', 'numeric'],  
    'harga_jual' => ['required', 'numeric'],  
];  
  
use Illuminate\Support\Facades\Validator;
```




```
$validator = Validator::make($request->all(), $rules);
    if ($validator->fails()) {
        return response()->json([
            'status' => false, json response, true: successful, false: failed
            'message' => 'Validation failed.',
            'msgField' => $validator->errors() indicates which field is an error
        ]);
    }

    $check = ItemModel::find($id);
    if ($check) {
        $check->update($request->all());
        return response()->json([
            'status' => true,
            'message' => 'Data was successfully updated'
        ]);
    } else{
        return response()->json([
            'status' => false,
            'message' => 'Data not found'
        ]);
    }
}

return redirect('/');

Public function confirm_ajax($id)
{
    $barang = ItemModel::find($id);
    return view('barang.confirm_ajax', ['item' => $barang]);
}

public function delete_ajax(Request $request, $id)
{
    if($request->ajax() || $request->wantsJson()){
        $barang = ItemModel::find($id);
        if($barang){ if it has been found
            $barang->delete(); Deleted items
            return response()->json([
                'status' => true,
                'message' => 'Data was successfully deleted'
            ]);
        }else{
            return response()->json([
                'status' => false,
                'message' => 'Data not found'
            ]);
        }
    }

    return redirect('/');

    public function import()
    {
        return view('item.import');
    }

    public function import_ajax(Request $request)
    {
        if($request->ajax() || $request->wantsJson()){
            $rules = [
                file validation must be xls orxlsx, max 1MB
                'file_barang' => ['required', 'mimes:XLSX', 'max:1024']
            ];

            $validator = Validator::make($request->all(), $rules);
            if($validator->fails()){
```



```
return response()->json([
    'status' => false,
    'message' => 'Validation Failed',
    'msgField' => $validator->errors()
]);
}

$file = $request->file('file_barang');    Retrieve files from requests

$reader = IOFactory::createReader('Xlsx');    Load Reader Excel File
$reader->setReadDataOnly(true);                Read only data
$spreadsheet = $reader->load($file->getRealPath());    Load Excel file
$sheet = $spreadsheet->getActiveSheet();            Take an active sheet

$data = $sheet->toArray(null, false, true, true);    Retrieve Excel Data

$insert = [];
if(count($data) > 1){    if the data is more than 1 line
    foreach ($data as $baris => $value) {
        if($baris > 1){    line to 1 is the header, then skip
            'kategori_id' => $value['A'],
            'barang_kode' => $value['B'],
            'barang_nama' => $value['C'],
            'harga_beli' => $value['D'],
            'harga_jual' => $value['E'],
            'created_at' => now(),
        ];
    }
}

if(count($insert) > 0){
    insert data into the database, if the data already exists, then it is
    ignored
    ItemModel::insertOrIgnore($insert);
}

return response()->json([
    'status' => true,
    'message' => 'Data imported successfully'
]);
}else{

    return response()->json([
        'status' => false,
        'message' => 'No data imported'
    ]);
}

return redirect('/');
```

8. Now let's try to run the browser and click on the import button on the Items menu



9. We upload the data template that we have prepared, and observe what happens.

Task 1 – Implementation of File Upload for Data Import:

1. Please implement practicum 1 in your respective projects for all menus
2. Observe and explain each stage you are working on, and describe it in the report
3. Submit the code for the implementation of Practice 1 in your github repository.

B. Export Excel di Laravel

The export to Excel feature in Laravel is a very useful tool in web applications that require professional data processing and presentation. This feature allows users to export data from databases or other processing results into Excel file format (.xlsx or .xls) that can be downloaded and used for various purposes, such as reporting, data analysis, or archival storage.

The export to Excel feature in Laravel allows you to provide data in a more flexible and easy-to-use format for end users.

Practicum 2 – Export Data to Excel

We will implement export data from the database to excel files in the Laravel project using the same library as practicum 1. The steps we work on are as follows:

1. We modify the `item/index.blade.php` by changing the following code

```
<a href="{{ url('/item/create') }}" class="btn btn-primary">Add Data</a>
```

We change it by

```
<a href="{{ url('/barang/export_excel') }}" class="btn btn-primary"><i class="fa fa-file-excel"></i> Export Barang</a>
```

We do this because **we no longer use** the Add Data button because we already use the **Add Data (Ajax) button**.

2. Then we add a route to the `route/web.php` to be able to process the excel export

```
Route::get('/barang/import',[BarangController::class,'import']); // ajax form upload excel
Route::post('/barang/import_ajax',[BarangController::class,'import_ajax']); // ajax import excel
Route::get('/barang/export_excel',[BarangController::class,'export_excel']); // export excel
```

3. Next we add the `export_excel()` function to the `BarangController.php` file
4. We take the data of the goods that we will export to excel (of course by including the relationship of the category of goods)



```
232     public function export_excel()  
233     {  
234         // ambil data barang yang akan di export  
235         $barang = BarangModel::select('kategori_id', 'barang_kode', 'barang_nama', 'harga_beli', 'harga_jual')  
236             ->orderBy('kategori_id')  
237             ->with('kategori')  
238             ->get();
```

5. Then we load the Spreadsheet library and we specify the data header on the first row in excel

```
240         // load library excel  
241         $spreadsheet = new \PhpOffice\PhpSpreadsheet\Spreadsheet();  
242         $sheet = $spreadsheet->getActiveSheet(); // ambil sheet yang aktif  
243  
244         $sheet->setCellValue('A1', 'No');  
245         $sheet->setCellValue('B1', 'Kode Barang');  
246         $sheet->setCellValue('C1', 'Nama Barang');  
247         $sheet->setCellValue('D1', 'Harga Beli');  
248         $sheet->setCellValue('E1', 'Harga Jual');  
249         $sheet->setCellValue('F1', 'Kategori');  
250  
251         $sheet->getStyle('A1:F1')->getFont()->setBold(true); // bold header
```

6. Next, we loop the data that we have obtained from the database, then we enter it into the excel cell

```
253         $no = 1; // nomor data dimulai dari 1  
254         $baris = 2; // baris data dimulai dari baris ke 2  
255         foreach ($barang as $key => $value) {  
256             $sheet->setCellValue('A' . $baris, $no);  
257             $sheet->setCellValue('B' . $baris, $value->barang_kode);  
258             $sheet->setCellValue('C' . $baris, $value->barang_nama);  
259             $sheet->setCellValue('D' . $baris, $value->harga_beli);  
260             $sheet->setCellValue('E' . $baris, $value->harga_jual);  
261             $sheet->setCellValue('F' . $baris, $value->kategori->kategori_nama); // ambil nama kategori  
262             $baris++;  
263             $no++;  
264         }
```

7. We set the width of each column in excel to match the length of the characters in each column

```
266         foreach(range('A', 'F') as $columnID) {  
267             $sheet->getColumnDimension($columnID)->setAutoSize(true); // set auto size untuk kolom  
268         }
```

8. The final part of the excel export process is that we set the name of the sheet, and the process to be downloadable by the user



```
270 $sheet->setTitle('Data Barang'); // set title sheet
271
272 $writer = IOFactory::createWriter($spreadsheet, 'Xlsx');
273 $filename = 'Data Barang '.date('Y-m-d H:i:s').'.xlsx';
274
275 header('Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet');
276 header('Content-Disposition: attachment;filename="'.$filename.'");
277 header('Cache-Control: max-age=0');
278 header('Cache-Control: max-age=1');
279 header('Expires: Mon, 26 Jul 1997 05:00:00 GMT');
280 header('Last-Modified: ' . gmdate('D, d M Y H:i:s') . ' GMT');
281 header('Cache-Control: cache, must-revalidate');
282 header('Pragma: public');
283
284 $writer->save('php://output');
285 exit;
286 } // end function export_excel
```

9. When it is complete, we try to download the Export file.

No	Kode Barang	Nama Barang	Harga Beli	Harga Jual	Kategori
1	SBK-001	Beras Cap Jago (5kg)	65000	68000	Sembako
2	SBK-002	Beras Bramo Cap Lele	80000	83000	Sembako
3	SBK-003	Telur Omega (10 butir)	22000	25000	Sembako
4	SNK-001	Happy Toss	10500	11000	Makanan Ringan
5	SNK-002	Oreo	7200	7800	Makanan Ringan
6	SNK-003	Sari Roti	11500	12500	Makanan Ringan
7	MND-001	Sabun Lifebouy	4250	5000	Peralatan Mandi
8	MND-002	Pasta Gigi Pepsoden	6750	7500	Peralatan Mandi
9	MND-003	Shampo Pantene	17500	18500	Peralatan Mandi
10	BAY-001	Susu SGM Coklat 900gr	92500	95000	Keperluan Bayi
11	BAY-002	Popok Mamy Poko	56000	58000	Keperluan Bayi
12	BAY-003	Baju Bayi 2th	89000	92500	Keperluan Bayi
13	MNM-001	Aqua 600ml	3700	4500	Minuman
14	MNM-002	Le Mineral	3500	4000	Minuman
15	MNM-003	Cleo 600ml	3750	4300	Minuman

Task 2 – Implementation of Excel Export Files:

1. Please implement practicum 2 in your respective projects for all menus
2. Observe and explain each stage you are working on, and describe it in the report
3. Submit the code for the implementation of Practum 2 in your github repository.

C. Export PDF di Laravel

The **export to PDF** feature in Laravel is very useful in situations where you need to produce documents that are structured, well-formatted, and easy to share. PDF (Portable Document Format) documents are one of the most commonly used formats for reports, invoices,



tickets, and various other official documents due to their stability and ability to maintain a consistent layout across multiple devices.

To implement export to PDF in Laravel, one of the most popular packages is **dompdf**. Using the **dompdf** package, we can easily convert the HTML view into a customizable PDF as needed. This feature is very useful in various contexts, such as creating invoices, reports, certificates, and other important documents.

Practicum 3 – Implementation of Export PDF in Laravel with **dompdf**

We'll apply export data to pdf files in the Laravel project using the **dompdf** library. The steps we work on are as follows:

1. We do the installation process of the **dompdf** library first by typing the command on the terminal/CMD

```
composer require barryvdh/laravel-dompdf
```

2. After the **dompdf** installation process is successful, then we add the following code to add the export to pdf button on the **item/index.blade.php**

```
<a href="{{ url('/barang/export_pdf') }}" class="btn btn-warning"><i class="fa fa-file-pdf"></i> Export Barang</a>
```

3. After that, we just need to fix the **route/web.php** for the pdf export process

```
Route::get('/barang/import',[BarangController::class,'import']); // ajax form upload excel
Route::post('/barang/import_ajax',[BarangController::class,'import_ajax']); // ajax import excel
Route::get('/barang/export_excel',[BarangController::class,'export_excel']); // export excel
Route::get('/barang/export_pdf',[BarangController::class,'export_pdf']); // export pdf
```

4. Next, we create the **export_pdf()** function on the **BarangController.php**

```
public function export_pdf()
{
    $barang = BarangModel::select('kategori_id','barang_kode','barang_nama','harga_beli','harga_jual')
        ->orderBy('kategori_id')
        ->orderBy('barang_kode')
        ->with('kategori')
        ->get();

    // use Barryvdh\DomPDF\Facade\Pdf;
    $pdf = Pdf::loadView('barang.export_pdf', ['barang' => $barang]);
    $pdf->setPaper('a4', 'portrait'); // set ukuran kertas dan orientasi
    $pdf->setOption("isRemoteEnabled", true); // set true jika ada gambar dari url
    $pdf->render();

    return $pdf->stream('Data Barang '.date('Y-m-d H:i:s').'.pdf');
}
```

5. Next, we create a view to be used as a pdf of the html layout. We can create a file with the name of the **item/export_pdf.blade.php**



INFO

Keep in mind, in export_pdf, we use CSS styles manually, and not using CSS frameworks like bootstraps. Because the dompdf library does not support all the styling features in the CSS framework.

6. Next, we create a view to generate html for the pdf view that we will present. View is in [item/export_pdf.blade.php](#)

```
<html>
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=utf-8"/>
</style>
  body{
    font-family: "Times New Roman", Times, serif;
    margins: 6px 20px 5px 20px;
    line-height: 15px;
  }

  Table {
    width:100%;
    border-collapse: collapse;
  }

  td, th {
    padding: 4px 3px;
  }

  th{
    text-align: left;
  }

  .d-block{
    display: block;
  }

  img.image{
    width: auto;
    height: 80px;
    max-width: 150px;
    max-height: 150px;
  }

  .text-right {
    text-align: right;
  }

  .text-center {
    text-align: center;
  }

  .p-1{
    padding: 5px 1px 5px 1px;
  }

  .font-10{
    font-size: 10pt;
  }

  .font-11{
    font-size: 11pt;
  }

  .font-12{
    font-size: 12pt;
  }

  .font-13{
    font-size: 13pt;
  }

  .border-bottom-header{
    border-bottom: 1px solid;
  }

  .border-all, .border-all th, .border-all td{
    border: 1px solid;
```



```
}
</style>
</head>
<body>
  <table class="border-bottom-header">
    <tr>
      <td width="15%" class="text-center"></td>
      <td width="85%">
        <span class="text-center d-block font-11 font-bold mb-1">MINISTRY OF
        EDUCATION, CULTURE, RESEARCH AND TECHNOLOGY</SPAN>
        <span class="text-center d-block font-13 font-bold mb-1">POLYTECHNIC NEGERI
        MALANG</span>
        <span class="text-center d-block font-10">Jl. Soekarno-Hatta No. 9 Malang
        65141</span>
        <span class="text-center d-block font-10">Phone (0341) 404424 Pes. 101-105,
        0341-404420, Fax. (0341) 404420</span>
        <span class="text-center d-block font-10">Page: www.polinema.ac.id</span>
      </td>
    </tr>
  </table>

  <h3 class="text-center">ITEM DATA REPORT</h3>
  <table class="border-all">
    <thead>
      <tr>
        <th class="text-center">No</th>
        <th>Goods Code</th>
        <th>Item Name</th>
        <th class="text-right">Purchase Price</th>
        <th class="text-right">Selling Price</th>
        <th>Category</th>
      </tr>
    </thead>
    <tbody>
      @foreach($barang as $b)
      <tr>
        <td class="text-center">{{ $loop->iteration }}</td>
        <td>{{ $b->barang_kode }}</td>
        <td>{{ $b->barang_nama }}</td>
        <td class="text-right">{{ number_format($b->harga_beli, 0, ',', '.') }}</td>
        <td class="text-right">{{ number_format($b->harga_jual, 0, ',', '.') }}</td>
        <td>{{ $b->kategori->kategori_nama }}</td>
      </tr>
      @endforeach
    </tbody>
  </table>
</body>
</html>
```

7. Next, we try to download the pdf export. Observe and learn...!!

Task 3 – Implement PDF Export on Laravel:

1. Please implement export pdf in your respective projects for all menus
2. Observe and explain each stage you are working on, and describe it in the report
3. Submit the code for the pdf export implementation in your github repository.



Task 4 – Implementation of Image File Upload:

1. Please implement the **file upload** feature to change **your profile photo** in your web project
2. Describe each stage you are working on, and describe it in the report
3. Submit the code for the pdf export implementation in your github repository.

*That's it, and happy learning ****